

SPACEGAME

Technical Comparisons and Details

Table of Contents

1. Purpose.....	2
2. Game Libraries and Engines.....	2
3. Code Editors and IDEs.....	2

1. Purpose

This document supports the SpaceGame analysis document. Its purpose is to record the technical reasoning behind the choice to build the prototype in C with raylib, instead of using a complete game engine or a lower-level multimedia library.

2. Game Libraries and Engines

Option	Fit for this challenge	Main advantage	Main limitation
C with raylib	Best fit	Keeps the project close to C programming while still handling window creation, input, textures and drawing.	Requires implementing game structure, collision logic and state management manually.
C with SDL	Possible but heavier	Gives more low-level control and is widely used for multimedia projects.	Would take more setup time for a first four-week game prototype.
Godot or Unity	Less suitable for this specific goal	A complete engine provides editors, scenes, physics, UI and export workflows.	The engine would hide some of the C/game-loop learning that I aim to do.
Custom engine	Not suitable	Would give maximum control and deep technical learning.	Too large for the assignment scope and would distract from making a playable prototype.

My choice: C with raylib lets me use my existing C knowledge while learning how a game loop, input and game states fit together. A complete engine would be useful for a project focused more on scene editing and ready-made systems. Building a custom engine would take too much time away from the playable game.

Sources: [raylib overview](#) | [SDL overview](#) | [Godot features](#) | [Unity concepts](#)

3. Code Editors and IDEs

Option	Main advantage	Main limitation	Fit for my project
Vim	Keyboard-focused editing that fits a terminal workflow. Can integrate build tools and GDB.	Learning modes and configuring extra assistance takes time.	My current choice because learning Vim and terminal tools is one of my goals.

Option	Main advantage	Main limitation	Fit for my project
Neovim	Similar editing approach, with a built-in language-server client for navigation and diagnostics.	The language server still needs to be installed and configured.	An alternative if I want more assistance while keeping a terminal workflow.
VS Code with C/C++ extension	Graphical editor with completion, error checking and debugging integration.	Needs a separate compiler and debugger, plus project configuration.	A practical alternative if editing or debugging starts delaying the game.
Full IDE such as CLion	Combines navigation, refactoring, project builds and debugging.	Adds an IDE interface and project or toolchain setup to learn. Too heavy for my laptop.	Useful when integrated assistance matters more than practising separate tools.
GNU nano	Modeless terminal editing with visible shortcut help and syntax highlighting.	Less support for understanding and navigating a growing C project.	Suitable for quick edits to text and configuration files.

My choice: I chose Vim to improve my editing and terminal skills while working in C. I need to keep the configuration manageable so it supports the game rather than becoming another project. Vim can work with a debugger; using it does not mean giving up debugging tools. The editor affects how I develop the game, but does not by itself make the compiled game faster.

Sources: [Vim and GDB](#) | [Neovim LSP](#) | [VS Code C/C++](#) | [CLion features](#) | [GNU nano manual](#)